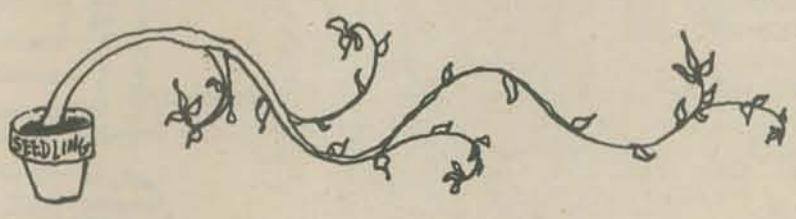
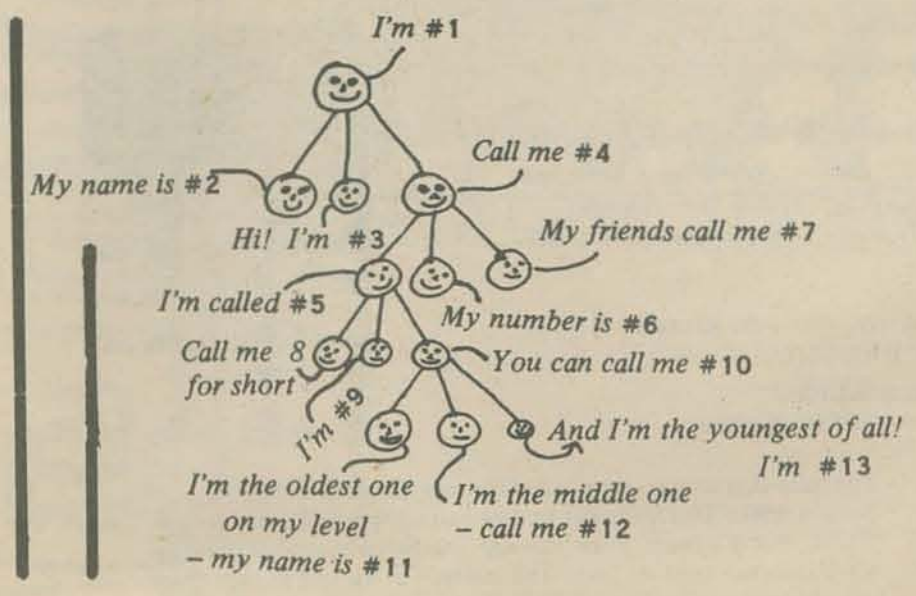
INSIDE THE CAVES
by dave of the caves

The CAVES family of games so far consists of CAVES1, CAVES2, CAVES3, PUBLIC CAVES.

You are invited to come inside the CAVES and see how they are constructed.



Last issue we left the **CAVES** looking like this:



In computer talk, this is called a **TREE** structure and the little circles are called **NODES**. The **CAVES** games all use **TREE** structures, pretending that each **NODE** is a cavern and each connection a tunnel.

The **CAVES1** program which constructed the **CAVES** in the drawing uses a core set of subroutines called **TREE SUBROUTINES**, which have a common entry point in line **7500**.

While building or moving through a **TREE**, you always **GOSUB 7500**. The subroutines are pretty clever -- by changing the value of **V1**, you can add **NODES** on any level, go up or down, look at the daughter **NODES** of your current **NODE** (one at a time), cut **NODES** off, prune them, or hop around the **TREE**.

And when the subroutines **RETURN**, *they will pass a value back in* **V2** *that tells you how things went.*

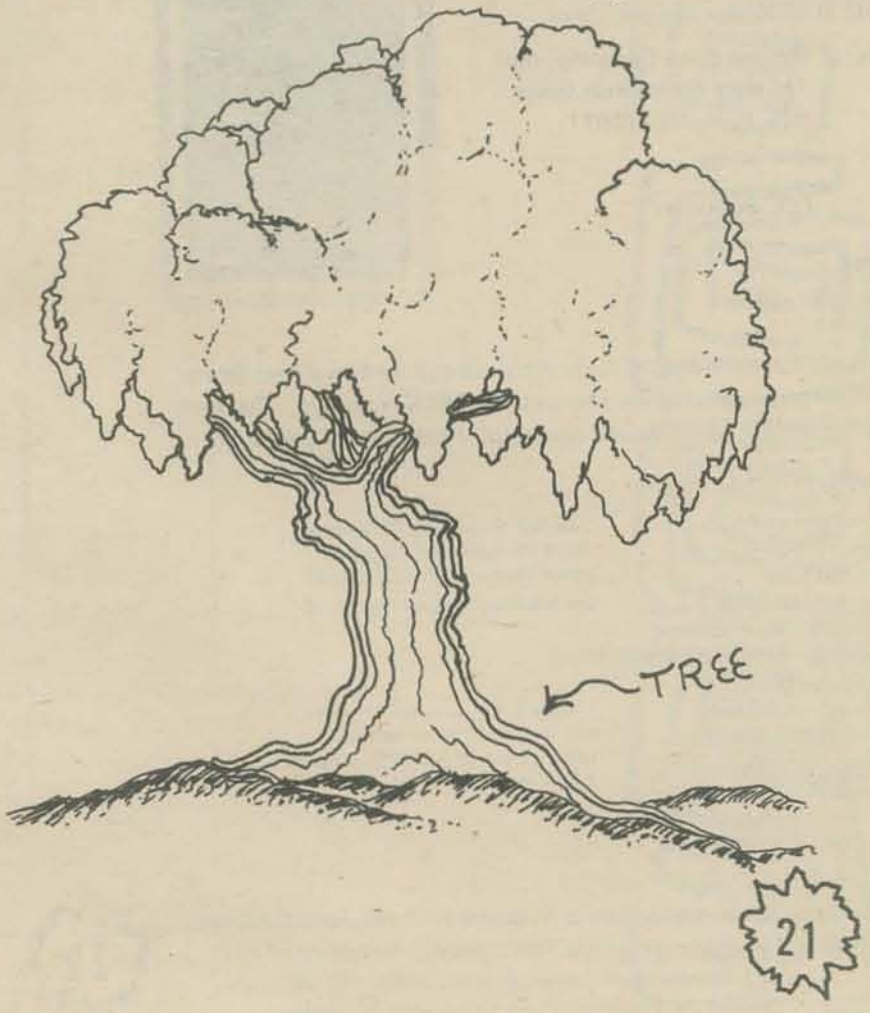
Now, without mincing words, I'll show you how **CAVES1** created the **TREE** structure at the beginning of this page:

```
10  V1=0                    initialize; make #1 the current NODE
20  GOSUB 7500              All TREE functions are accomplished
                            by GOSUB 7500 -- V1 determines
                            which functions

30  D1=N1
40  FOR L=1 TO 4
50  FOR D=1 TO 3
60  D1=D1+1                 pick a new name for next daughter
70  V1=1                    this functions adds D1 as next
                            daughter

80  GOSUB 7500
90  NEXT D
100 D1=D1-INT(3*RND(0))     pick one of the last 3 NODES
110 V1=6                    this function goes down to D1
120 GOSUB 7500
130 NEXT L
140 O=N1                    Since each L loop moves you down
                            to a random daughter NODE, use the
                            last one as the Out NODE
```

Next issue I'll show you how to use **TREE SUBROUTINES** in your own programs and games! See ya !!!

V1 and V2

```
7400  REM *** TREE SUBROUTINES BEGIN ***
7402  REM SACRED NAMES: N(127),P(385),L(99),N9,P1,P2,P3,P4,P5,P9,
7404  REM                L1,FNP,FNN
7406  REM ARGUMENT: V1,V2,N1,D1
7408  REM N1 IS NAME OF CURRENT NODE (FOR TOP NODE, N1=1)
7410  REM D1 IS NAME OF DAUGHTER NODE
7412  REM NODE NAMES CAN BE ANY NUMBER EXCEPT 9999
7414  REM
7416  REM V1=  0 INITIALIZE TREE; MAKE #1 THE CURRENT NODE
7418  REM      1 ADD D1 AS NEXT DAUGHTER TO CURRENT NODE
7420  REM      2 CUT D1 OFF FROM CURRENT NODE (BUT LEAVE IT AND
7422  REM        ALL BELOW IT)
7424  REM      3 PRUNE D1 (AND ALL BELOW IT) FROM TREE
7426  REM      4 GO UP FROM N1 TO MOTHER NODE (UNLESS AT TOP)
7428  REM      5 GO UP FROM N1 THE WAY YOU CAME (UNLESS AT TOP)
7430  REM      6 GO DOWN TO D1 FROM CURRENT NODE (IF LEGAL)
7432  REM      7 RETURN WITH NEXT DAUGHTER NODE (AFTER D1) IN D1
7434  REM        IF NO MORE, D1=9999
7436  REM        IF D1 ISN'T A DAUGHTER OF CURRENT NODE, RETURN WITH
7438  REM        FIRST DAUGHTER NODE
7440  REM      8 RESET CURRENT NODE TO D1, WHEREVER IT IS IN TREE
7442  REM
7444  REM V2=  2  INSTRUCTION CARRIED OUT, BUT D1 IS A NEW NODE NAME
7446  REM      1  INSTRUCTION CARRIED OUT
7448  REM     -1 CANNOT MOVE UP (OR DOWN) BECAUSE ON TOP (BOTTOM)
7450  REM     -2 MORE THAN 99 NODES
7452  REM     -3 TOO MANY CONNECTIONS
7454  REM     -4 D1 IS NOT A NODE NAME
7480  DIM N(127),P(385),L(99)
7482  DEF FNP(X)=INT(P(X)/512)
7484  DEF FNN(X)=P(X)-INT(P(X)/512)*512
7500  REM ***ENTRY POINT FOR TREE SUBROUTINES
```